I/O in Java: Basics, Console I/O, Reading & Writing Files (Complete File Handling)

Everything that is part of Java file handling—**every class**, **every method**, **every operation**, **every style** (old, intermediate, modern, NIO.2) is covered.

🌟 1. I/O BASICS (Streams in Java)

Java uses **streams** to handle input and output.

Types of Streams

Type	Description	Examples
Byte Streams	Read/write raw bytes	`FileInputStream`, `FileOutputStream`
Character Streams	Read/write text (Unicode-safe)	`FileReader`, `FileWriter`
Buffered Streams	Faster I/O using internal buffer	`BufferedReader`, `BufferedWriter`
Data Streams	Read/write primitive datatypes	`DataInputStream`, `DataOutputStream`
Object Streams	Read/write full objects	`ObjectInputStream`, `ObjectOutputStream`
NIO Streams	High-performance, asynchronous	`Files`, `Path`, `Channels`

🌟 2. Console I/O (User Input & Output)

2.1 Reading Input from Console

✔ Using `Scanner`

```
Scanner sc = new Scanner(System.in);
String name = sc.nextLine();
int age = sc.nextInt();
```

✔ Using `BufferedReader`

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String s = br.readLine();
```

✔ Using `Console` Class (Secure Inputs)

```
Console c = System.console();
String user = c.readLine();
char[] pass = c.readPassword();
```

★ 3. FILE HANDLING (All Methods & All Classes)

✔ 3.1 Checking & Inspecting Files (`File` Class)

```
File f = new File("data.txt");

f.exists();
f.createNewFile();
f.isFile();
f.isDirectory();
f.length();
f.delete();
```

★ 4. READING FILES (All Techniques)

✔ Method 1: FileInputStream (Byte Stream)

```
FileInputStream fis = new FileInputStream("a.txt");
int b;
while((b = fis.read()) != -1){
    System.out.print((char)b);
}
fis.close();
```

✔ Method 2: FileReader (Character Stream)

```
FileReader fr = new FileReader("a.txt");
int c;
while((c = fr.read()) != -1) {
    System.out.print((char)c);
}
fr.close();
```

✔ Method 3: BufferedReader (Fast + readLine())

```
BufferedReader br = new BufferedReader(new FileReader("a.txt"));
String line;
while((line = br.readLine()) != null){
    System.out.println(line);
}
br.close();
```

✔ Method 4: Scanner (Easy Reading)

```
Scanner sc = new Scanner(new File("a.txt"));
while(sc.hasNextLine()){
    System.out.println(sc.nextLine());
}
sc.close();
```

✔ Method 5: NIO Files.readAllLines()

(Shortest + Modern)

```
List<String> lines = Files.readAllLines(Path.of("a.txt"));
lines.forEach(System.out::println);
```

✔ Method 6: Reading as Bytes

```
byte[] data = Files.readAllBytes(Path.of("img.png"));
```

✔ Method 7: Using FileChannel

```
FileChannel ch = FileChannel.open(Path.of("a.txt"));
ByteBuffer buffer = ByteBuffer.allocate(1024);
ch.read(buffer);
```

🌟 5. WRITING TO FILES (All Techniques)

✔ Method 1: FileOutputStream (Bytes)

```
FileOutputStream fos = new FileOutputStream("a.txt");
fos.write("Hello".getBytes());
fos.close();
```

✔ Method 2: FileWriter (Characters)

```
FileWriter fw = new FileWriter("a.txt");
fw.write("Hello Java!");
fw.close();
```

✔ Method 3: BufferedWriter

```
BufferedWriter bw = new BufferedWriter(new FileWriter("a.txt"));
bw.write("Line 1");
bw.newLine();
bw.write("Line 2");
bw.close();
```

✔ Method 4: PrintWriter (Best for Text Files)

```
PrintWriter pw = new PrintWriter("a.txt");
pw.println("Hello");
pw.printf("%d %s", 10, "hi");
pw.close();
```

✔ Method 5: NIO Files.write()

```
Files.write(Path.of("a.txt"), "Hello NIO".getBytes());
```

✔ Method 6: Appending to File

```
Files.write(
    Path.of("a.txt"),
    "New Line\n".getBytes(),
    StandardOpenOption.APPEND
```

```
);
```

✔ Method 7: FileChannel (High-Speed Writing)

```
FileChannel ch = FileChannel.open(Path.of("log.txt"),
        StandardOpenOption.WRITE,
        StandardOpenOption.CREATE);

ByteBuffer buf = ByteBuffer.wrap("Hello".getBytes());
ch.write(buf);
```

★ 6. OBJECT SERIALIZATION (Read/Write Objects)

✔ Writing an Object

```
ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("obj.dat"));
oos.writeObject(studentObject);
oos.close();
```

✔ Reading an Object

```
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("obj.dat"));
Student s = (Student) ois.readObject();
ois.close();
```

★ 7. RANDOM ACCESS FILE

Read/write anywhere in the file.

```
RandomAccessFile raf = new RandomAccessFile("data.dat", "rw");

raf.writeInt(100);
raf.seek(0);
System.out.println(raf.readInt());

raf.close();
```

⭐ 8. DIRECTORY + FILE OPERATIONS (NIO)

Create Directory

```
Files.createDirectory(Path.of("myfolder"));
```

List Files

```
Files.list(Path.of(".")).forEach(System.out::println);
```

Copy File

```
Files.copy(Path.of("a.txt"), Path.of("b.txt"));
```

Move File

```
Files.move(Path.of("a.txt"), Path.of("folder/a.txt"));
```

Delete File

```
Files.delete(Path.of("a.txt"));
```

⭐ 9. TRY-WITH-RESOURCES (Modern Best Practice)

Automatically closes the stream.

```
try (BufferedReader br = new BufferedReader(new FileReader("a.txt"))) {
    br.lines().forEach(System.out::println);
}
```

⭐ 10. FILE HANDLING TOPICS FOR EXAMS & INTERVIEWS

✔ **Buffered vs Unbuffered Streams**

- ✔ **Byte vs Character Streams**
- ✔ **Decorator Pattern (Used in I/O)**
- ✔ **Serialization & `transient` keyword**
- ✔ **NIO vs IO Performance**
- ✔ **try-with-resources**
- ✔ **RandomAccessFile use**
- ✔ **File permissions handling**
- ✔ **Path vs File difference**

🌟 11. COMPLETE JAVA FILE HANDLING CHEATSHEET (SUMMARY)

Task	Recommended API
Read text lines	`BufferedReader` or `Files.readAllLines()`
Write text	`BufferedWriter`, `PrintWriter`
Read/write binary	`FileInputStream`, `FileOutputStream`
Fast I/O	`BufferedStreams`
Object I/O	`ObjectOutputStream`
Directory operations	`Files` (NIO)
High-performance I/O	`FileChannel`, `ByteBuffer`
Easy user input	`Scanner`
